

```
// 使用之前设定的书店程序异常类
Sales_data item1, item2, sum;
while (cin >> item1 >> item2) {           // 读取两条交易信息
    try {
        sum = item1 + item2;                // 计算它们的和
        // 此处使用 sum
    } catch (const isbn_mismatch &e) {
        cerr << e.what() << ": left isbn(" << e.left
            << ") right isbn(" << e.right << ") " << endl;
    }
}
```

18.1.5 节练习

785

练习 18.9: 定义本节描述的书店程序异常类，然后为 `Sales_data` 类重新编写一个复合赋值运算符并令其抛出一个异常。

练习 18.10: 编写程序令其对两个 ISBN 编号不相同的对象执行 `Sales_data` 的加法运算。为该程序编写两个不同的版本：一个处理异常，另一个不处理异常。观察并比较这两个程序的行为，用心体会当出现了一个未被捕获的异常时程序会发生什么情况。

练习 18.11: 为什么 `what` 函数不应该抛出异常？

18.2 命名空间

大型程序往往会使用多个独立开发的库，这些库又会定义大量的全局名字，如类、函数和模板等。当应用程序用到多个供应商提供的库时，不可避免地会发生某些名字相互冲突的情况。多个库将名字放置在全局命名空间中将引发**命名空间污染**（`namespace pollution`）。

传统上，程序员通过将其定义的全局实体名字设得很长来避免命名空间污染问题，这样的名字中通常包含表示名字所属库的前缀部分：

```
class cplusplus_primer_Query { ... };
string cplusplus_primer_make_plural(size_t, string&);
```

这种解决方案显然不太理想：对于程序员来说，书写和阅读这么长的名字费时费力且过于烦琐。

命名空间（`namespace`）为防止名字冲突提供了更加可控的机制。命名空间分割了全局命名空间，其中每个命名空间是一个作用域。通过在某个命名空间中定义库的名字，库的作者（以及用户）可以避免全局名字固有的限制。

18.2.1 命名空间定义

一个命名空间的定义包含两部分：首先是关键字 `namespace`，随后是命名空间的名字。在命名空间名字后面是一系列由花括号括起来的声明和定义。只要能出现在全局作用域中的声明就能置于命名空间内，主要包括：类、变量（及其初始化操作）、函数（及其定义）、模板和其他命名空间：

```
namespace cplusplus_primer {
    class Sales_data { /* ... */};
}
```

```

Sales_data operator+(const Sales_data&,
                    const Sales_data&);

class Query { /* ... */ };
class Query_base { /* ... */ };
} // 命名空间结束后无须分号, 这一点与块类似

```

786 上面的代码定义了一个名为 `cplusplus_primer` 的命名空间, 该命名空间包含四个成员: 三个类和一个重载的`+`运算符。

和其他名字一样, 命名空间的名字也必须在定义它的作用域内保持唯一。命名空间既可以定义在全局作用域内, 也可以定义在其他命名空间中, 但是不能定义在函数或类的内部。



Note 命名空间作用域后面无须分号。

每个命名空间都是一个作用域

和其他作用域类似, 命名空间中的每个名字都必须表示该空间内的唯一实体。因为不同命名空间的作用域不同, 所以在不同命名空间内可以有相同名字的成员。

定义在某个命名空间中的名字可以被该命名空间内的其他成员直接访问, 也可以被这些成员内嵌作用域中的任何单位访问。位于该命名空间之外的代码则必须明确指出所用的名字属于哪个命名空间:

```

cplusplus_primer::Query q =
    cplusplus_primer::Query("hello");

```

如果其他命名空间 (比如说 `AddisonWesley`) 也提供了一个名为 `Query` 的类, 并且我们希望使用这个类替代 `cplusplus_primer` 中定义的同名类, 则可以按照如下方式修改代码:

```

AddisonWesley::Query q = AddisonWesley::Query("hello");

```

命名空间可以是不连续的

如我们在 16.5 节 (第 626 页) 介绍过的, 命名空间可以定义在几个不同的部分, 这一点与其他作用域不太一样。编写如下的命名空间定义:

```

namespace nsp {
// 相关声明
}

```

可能是定义了一个名为 `nsp` 的新命名空间, 也可能是为已经存在的命名空间添加一些新成员。如果之前没有名为 `nsp` 的命名空间定义, 则上述代码创建一个新的命名空间; 否则, 上述代码打开已经存在的命名空间定义并为其添加一些新成员的声明。

命名空间的定义可以不连续的特性使得我们可以将几个独立的接口和实现文件组成一个命名空间。此时, 命名空间的组织方式类似于我们管理自定义类及函数的方式:

- 命名空间的一部分成员的作用是定义类, 以及声明作为类接口的函数及对象, 则这些成员应该置于头文件中, 这些头文件将被包含在使用了这些成员的文件中。
- 命名空间成员的定义部分则置于另外的源文件中。

787 在程序中某些实体只能定义一次: 如非内联函数、静态数据成员、变量等, 命名空间中定义的名字也需要满足这一要求, 我们可以通过上面的方式组织命名空间并达到目的。这种接口和实现分离的机制确保我们所需的函数和其他名字只定义一次, 而只要是用到这些实

体的地方都能看到对于实体名字的声明。



定义多个类型不相关的命名空间应该使用单独的文件分别表示每个类型（或关联类型构成的集合）。

定义本书的命名空间

通过使用上述接口与实现分离的机制，我们可以将 `cplusplus_primer` 库定义在几个不同的文件中。`Sales_data` 类的声明及其函数将置于 `Sales_data.h` 头文件中，第 15 章介绍的 `Query` 类将置于 `Query.h` 头文件中，以此类推。对应的实现文件将分别是 `Sales_data.cc` 和 `Query.cc`：

```
// --- Sales_data.h ---
// #include 应该出现在打开命名空间的操作之前
#include <string>
namespace cplusplus_primer {
    class Sales_data { /* ... */};
    Sales_data operator+(const Sales_data&,
                        const Sales_data&);
    // Sales_data 的其他接口函数的声明
}
// --- Sales_data.cc ---
// 确保#include 出现在打开命名空间的操作之前
#include "Sales_data.h"

namespace cplusplus_primer {
    // Sales_data 成员及重载运算符的定义
}
```

程序如果想使用我们定义的库，必须包含必要的头文件，这些头文件中的名字定义在命名空间 `cplusplus_primer` 内：

```
// --- user.cc ---
// Sales_data.h 头文件的名字位于命名空间 cplusplus_primer 中
#include "Sales_data.h"
int main()
{
    using cplusplus_primer::Sales_data;
    Sales_data transl, trans2;
    // ...
    return 0;
}
```

这种程序的组织方式提供了开发者和库用户所需的模块性。每个类仍组织在自己的接口和实现文件中，一个类的用户不必编译与其他类相关的名字。我们对用户隐藏了实现细节，同时允许文件 `Sales_data.cc` 和 `user.cc` 被编译并链接成一个程序而不会产生任何编译时错误或链接时错误。库的开发者可以分别实现每一个类，相互之间没有干扰。

有一点需要注意，在通常情况下，我们不把 `#include` 放在命名空间内部。如果我们这么做了，隐含的意思是把头文件中所有的名字定义成该命名空间的成员。例如，如果 `Sales_data.h` 在包含 `string` 头文件前就已经打开了命名空间 `cplusplus_primer`，则程序将出错，因为这么做意味着我们试图将命名空间 `std` 嵌套在命名空间 `cplusplus_primer` 中。

定义命名空间成员

假定作用域中存在合适的声明语句，则命名空间中的代码可以使用同一命名空间定义的名字的简写形式：

```
#include "Sales_data.h"
namespace cplusplus_primer {      // 重新打开命名空间 cplusplus_primer
// 命名空间中定义的成员可以直接使用名字，此时无须前缀
std::istream&
operator>>(std::istream& in, Sales_data& s) { /* ... */ }
}
```

也可以在命名空间定义的外部定义该命名空间的成员。命名空间对于名字的声明必须在作用域内，同时该名字的定义需要明确指出其所属的命名空间：

```
// 命名空间之外定义的成员必须使用含有前缀的名字
cplusplus_primer::Sales_data
cplusplus_primer::operator+(const Sales_data& lhs,
                             const Sales_data& rhs)
{
    Sales_data ret(lhs);
    // ...
}
```

和定义在类外部的类成员一样，一旦看到含有完整前缀的名字，我们就可以确定该名字位于命名空间的作用域内。在命名空间 `cplusplus_primer` 内部，我们可以直接使用该命名空间的其他成员，比如在上面的代码中，可以直接使用 `Sales_data` 定义函数的形参。

尽管命名空间的成员可以定义在命名空间外部，但是这样的定义必须出现在所属命名空间的外层空间中。换句话说，我们可以在 `cplusplus_primer` 或全局作用域中定义 `Sales_data operator+`，但是不能在一个不相关的作用域中定义这个运算符。

模板特例化

模板特例化必须定义在原始模板所属的命名空间中（参见 16.5 节，第 626 页）。和其他命名空间名字类似，只要我们在命名空间中声明了特例化，就能在命名空间外部定义它了：

```
// 我们必须将模板特例化声明成 std 的成员
namespace std {
    template <> struct hash<Sales_data>;
}
// 在 std 中添加了模板特例化的声明后，就可以在命名空间 std 的外部定义它了
template <> struct std::hash<Sales_data>
{
    size_t operator()(const Sales_data& s) const
    { return hash<string>()(s.bookNo) ^
        hash<unsigned>()(s.units_sold) ^
        hash<double>()(s.revenue); }
    // 其他成员与之前的版本一致
};
```

全局命名空间

全局作用域中定义的名字（即在所有类、函数及命名空间之外定义的名字）也就是定义在全局命名空间（`global namespace`）中。全局命名空间以隐式的方式声明，并且在所有

程序中都存在。全局作用域中定义的名字被隐式地添加到全局命名空间中。

作用域运算符同样可以用于全局作用域的成员，因为全局作用域是隐式的，所以它并没有名字。下面的形式

```
::member_name
```

表示全局命名空间中的一个成员。

嵌套的命名空间

嵌套的命名空间是指定义在其他命名空间中的命名空间：

```
namespace cplusplus_primer {
    // 第一个嵌套的命名空间：定义了库的 Query 部分
    namespace QueryLib {
        class Query { /* ... */ };
        Query operator&(const Query&, const Query&);
        // ...
    }
    // 第二个嵌套的命名空间：定义了库的 Sales_data 部分
    namespace Bookstore {
        class Quote { /* ... */ };
        class Disc_quote : public Quote { /* ... */ };
        // ...
    }
}
```

上面的代码将命名空间 `cplusplus_primer` 分割为两个嵌套的命名空间，分别是 `QueryLib` 和 `Bookstore`。

嵌套的命名空间同时是一个嵌套的作用域，它嵌套在外层命名空间的作用域中。嵌套的命名空间中的名字遵循的规则与往常类似：内层命名空间声明的名字将隐藏外层命名空间声明的同名成员。在嵌套的命名空间中定义的名字只在内层命名空间中有效，外层命名空间中的代码要想访问它必须在名字前添加限定符。例如，在嵌套的命名空间 `QueryLib` 中声明的类名是

```
cplusplus_primer::QueryLib::Query
```

内联命名空间

C++11 新标准引入了一种新的嵌套命名空间，称为**内联命名空间**（`inline namespace`）。和普通的嵌套命名空间不同，内联命名空间中的名字可以被外层命名空间直接使用。也就是说，我们无须在内联命名空间的名字前添加表示该命名空间的前缀，通过外层命名空间的名字就可以直接访问它。

定义内联命名空间的方式是在关键字 `namespace` 前添加关键字 `inline`：

```
inline namespace FifthEd {
    // 该命名空间表示本书第 5 版的代码
}
namespace FifthEd { // 隐式内联
    class Query_base { /* ... */ };
    // 其他与 Query 有关的声明
}
```

790

C++
11


```
// 二义性: i 的定义既出现在全局作用域中, 又出现在未嵌套的未命名的命名空间中
i = 10;
```

其他情况下, 未命名的命名空间中的成员都属于正确的程序实体。和所有命名空间类似, 一个未命名的命名空间也能嵌套在其他命名空间当中。此时, 未命名的命名空间中的成员可以通过外层命名空间的名字来访问:

```
namespace local {
    namespace {
        int i;
    }
}
// 正确: 定义在嵌套的未命名的命名空间中的 i 与全局作用域中的 i 不同
local::i = 42;
```

未命名的命名空间取代文件中的静态声明

792

在标准 C++ 引入命名空间的概念之前, 程序需要将名字声明成 `static` 的以使得其对于整个文件有效。在文件中进行静态声明的做法是从 C 语言继承而来的。在 C 语言中, 声明为 `static` 的全局实体在其所在的文件外不可见。



在文件中进行静态声明的做法已经被 C++ 标准取消了, 现在的做法是使用未命名的命名空间。

18.2.1 节练习

练习 18.12: 将你为之前各章练习编写的程序放置在各自的命名空间中。也就是说, 命名空间 `chapter15` 包含 `Query` 程序的代码, 命名空间 `chapter10` 包含 `TextQuery` 的代码; 使用这种结构重新编译 `Query` 代码示例。

练习 18.13: 什么时候应该使用未命名的命名空间?

练习 18.14: 假设下面的 `operator*` 声明的是嵌套的命名空间 `mathLib::MatrixLib` 的一个成员:

```
namespace mathLib {
    namespace MatrixLib {
        class matrix { /* ... */ };
        matrix operator*
            (const matrix &, const matrix &);
        // ...
    }
}
```

请问你应该如何在全局作用域中声明该运算符?

18.2.2 使用命名空间成员

像 `namespace_name::member_name` 这样使用命名空间的成员显然非常烦琐, 特别是当命名空间的名字很长时尤其如此。幸运的是, 我们可以通过一些其他更简便的方法使用命名空间的成员。之前的程序已经使用过其中一种方法, 即 `using` 声明 (参见 3.1 节, 第 74 页)。本节还将介绍另外几种方法, 如命名空间的别名以及 `using` 指示等。

命名空间的别名

命名空间的别名 (namespace alias) 使得我们可以为命名空间的名字设定一个短得多的同义词。例如, 一个很长的命名空间的名字形如

```
namespace cplusplus_primer { /* ... */ };
```

我们可以为其设定一个短得多的同义词:

```
namespace primer = cplusplus_primer;
```

793

命名空间的别名声明以关键字 `namespace` 开始, 后面是别名所用的名字、= 符号、命名空间原来的名字以及一个分号。不能在命名空间还没有定义前就声明别名, 否则将产生错误。

命名空间的别名也可以指向一个嵌套的命名空间:

```
namespace Qlib = cplusplus_primer::QueryLib;  
Qlib::Query q;
```



一个命名空间可以有好几个同义词或别名, 所有别名都与命名空间原来的名字等价。

using 声明: 扼要概述

一条 **using 声明** (using declaration) 语句一次只引入命名空间的一个成员。它使得我们可以清楚地知道程序中所用的到底是哪个名字。

using 声明引入的名字遵守与过去一样的作用域规则: 它的有效范围从 using 声明的地方开始, 一直到 using 声明所在的作用域结束为止。在此过程中, 外层作用域的同名实体将被隐藏。未加限定的名字只能在 using 声明所在的作用域以及其内层作用域中使用。在有效作用域结束后, 我们就必须使用完整的经过限定的名字了。

一条 using 声明语句可以出现在全局作用域、局部作用域、命名空间作用域以及类的作用域中。在类的作用域中, 这样的声明语句只能指向基类成员 (参见 15.5 节, 第 546 页)。

using 指示

using 指示 (using directive) 和 using 声明类似的地方是, 我们可以使用命名空间名字的简写形式; 和 using 声明不同的地方是, 我们无法控制哪些名字是可见的, 因为所有名字都是可见的。

using 指示以关键字 `using` 开始, 后面是关键字 `namespace` 以及命名空间的名字。如果这里所用的名字不是一个已经定义好的命名空间的名字, 则程序将发生错误。using 指示可以出现在全局作用域、局部作用域和命名空间作用域中, 但是不能出现在类的作用域中。

using 指示使得某个特定的命名空间中所有的名字都可见, 这样我们就无须再为它们添加任何前缀限定符了。简写的名字从 using 指示开始, 一直到 using 指示所在的作用域结束都能使用。



如果我们提供了一个对 `std` 等命名空间的 using 指示而未做任何特殊控制的话, 将重新引入由于使用了多个库而造成的名字冲突问题。

using 指示与作用域

using 指示引入的名字的作用域远比 using 声明引入的名字的作用域复杂。如我们所知, using 声明的名字的作用域与 using 声明语句本身的作用域一致, 从效果上看就好像 using 声明语句为命名空间的成员在当前作用域内创建了一个别名一样。

using 指示所做的绝非声明别名这么简单。相反, 它具有将命名空间成员提升到包含命名空间本身和 using 指示的最近作用域的能力。

794

using 声明和 using 指示在作用域上的区别直接决定了它们工作方式的不同。对于 using 声明来说, 我们只是简单地令名字在局部作用域内有效。相反, using 指示是令整个命名空间的所有内容变得有效。通常情况下, 命名空间中会含有一些不能出现在局部作用域中的定义, 因此, using 指示一般被看作是出现在最近的外层作用域中。

在最简单的情况下, 假定我们有一个命名空间 A 和一个函数 f, 它们都定义在全局作用域中。如果 f 含有一个对 A 的 using 指示, 则在 f 看来, A 中的名字仿佛是出现在全局作用域中 f 之前的位置一样:

```
// 命名空间 A 和函数 f 定义在全局作用域中
namespace A {
    int i, j;
}
void f()
{
    using namespace A;           // 把 A 中的名字注入到全局作用域中
    cout << i * j << endl;      // 使用命名空间 A 中的 i 和 j
    // ...
}
```

using 指示示例

让我们看一个简单的示例:

```
namespace blip {
    int i = 16, j = 15, k = 23;
    // 其他声明
}
int j = 0;           // 正确: blip 的 j 隐藏在命名空间中
void manip()
{
    // using 指示, blip 中的名字被“添加”到全局作用域中
    using namespace blip; // 如果使用了 j, 则将在::j 和 blip::j 之间产生冲突
    ++i;                  // 将 blip::i 设定为 17
    ++j;                  // 二义性错误: 是全局的 j 还是 blip::j?
    ++::j;                // 正确: 将全局的 j 设定为 1
    ++blip::j;            // 正确: 将 blip::j 设定为 16
    int k = 97;           // 当前局部的 k 隐藏了 blip::k
    ++k;                  // 将当前局部的 k 设定为 98
}
```

manip 的 using 指示使得程序可以直接访问 blip 的所有名字, 也就是说, manip 的代码可以使用 blip 中名字的简写形式。

blip 的成员看起来好像是定义在 blip 和 manip 所在的作用域一样。假定 manip

795

定义在全局作用域中，则 blip 的成员也好像是定义在全局作用域中一样。

当命名空间被注入到它的外层作用域之后，很有可能该命名空间中定义的名字会与其外层作用域中的成员冲突。例如在 manip 中，blip 的成员 j 就与全局作用域中的 j 产生了冲突。这种冲突是允许存在的，但是要想使用冲突的名字，我们就必须明确指出名字的版本。manip 中所有未加限定的 j 都会产生二义性错误。

为了使用像 j 这样的名字，我们必须使用作用域运算符来明确指出所需的版本。我们使用 ::j 来表示定义在全局作用域中的 j，而使用 blip::j 来表示定义在 blip 中的 j。

因为 manip 的作用域和命名空间的作用域不同，所以 manip 内部的声明可以隐藏命名空间中的某些成员名字。例如，局部变量 k 隐藏了命名空间的成员 blip::k。在 manip 内使用 k 不存在二义性，它指的就是局部变量 k。

头文件与 using 声明或指示

头文件如果在其顶层作用域中含有 using 指示或 using 声明，则会将名字注入到所有包含了该头文件的文件中。通常情况下，头文件应该只负责定义接口部分的名字，而不定义实现部分的名字。因此，头文件最多只能在它的函数或命名空间内使用 using 指示或 using 声明（参见 3.1 节，第 75 页）。

提示：避免 using 指示

using 指示一次性注入某个命名空间的所有名字，这种用法看似简单实则充满了风险：只使用一条语句就突然将命名空间中所有成员的名字变得可见了。如果应用程序使用了多个不同的库，而这些库中的名字通过 using 指示变得可见，则全局命名空间污染的问题将重新出现。

而且，当引入库的新版本后，正在工作的程序很可能会编译失败。如果新版本引入了一个与应用程序正在使用的名字冲突的名字，就会出现这个问题。

另一个风险是由 using 指示引发的二义性错误只有在使用了冲突名字的地方才能被发现。这种延后的检测意味着可能在特定库引入很久之后才爆发冲突。直到程序开始使用该库的新部分后，之前一直未被检测到的错误才会出现。

相比于使用 using 指示，在程序中对命名空间的每个成员分别使用 using 声明效果更好，这么做可以减少注入到命名空间中的名字数量。using 声明引起的二义性问题在声明处就能发现，无须等到使用名字的地方，这显然对检测并修改错误大有益处。



using 指示也并非一无是处，例如在命名空间本身的实现文件中就可以使用 using 指示。

18.2.2 节练习

练习 18.15：说明 using 指示与 using 声明的区别。

练习 18.16：假定在下面的代码中标记为“位置 1”的地方是对于命名空间 Exercise 中所有成员的 using 声明，请解释代码的含义。如果这些 using 声明出现在“位置 2”又会怎样呢？将 using 声明变为 using 指示，重新回答之前的问题。

```
namespace Exercise {  
    int ivar = 0;  
    double dvar = 0;
```

```

    const int limit = 1000;
}
int ivar = 0;
// 位置 1
void manip() {
    // 位置 2
    double dvar = 3.1416;
    int iobj = limit + 1;
    ++ivar;
    ++::ivar;
}

```

练习 18.17：实际编写代码检验你对上一题的回答是否正确。

18.2.3 类、命名空间与作用域

对命名空间内部名字的查找遵循常规的查找规则：即由内向外依次查找每个外层作用域。外层作用域也可能是一个或多个嵌套的命名空间，直到最外层的全局命名空间查找过程终止。只有位于开放的块中且在使用点之前声明的名字才被考虑：

```

namespace A {
    int i;
    namespace B {
        int i;           // 在 B 中隐藏了 A::i
        int j;
        int f1()
        {
            int j;       // j 是 f1 的局部变量，隐藏了 A::B::j
            return i;     // 返回 B::i
        }
    } // 命名空间 B 结束，此后 B 中定义的名字不再可见
    int f2() {
        return j;        // 错误：j 没有被定义
    }
    int j = i;           // 用 A::i 进行初始化
}

```

对于位于命名空间中的类来说，常规的查找规则仍然适用：当成员函数使用某个名字时，首先在该成员中进行查找，然后在类中查找（包括基类），接着在外层作用域中查找，这时一个或几个外层作用域可能就是命名空间：

797

```

namespace A {
    int i;
    int k;
    class C1 {
    public:
        C1(): i(0), j(0) { } // 正确：初始化 C1::i 和 C1::j
        int f1() { return k; } // 返回 A::k
        int f2() { return h; } // 错误：h 未定义
        int f3();
    private:
        int i;                // 在 C1 中隐藏了 A::i
    }
}

```

```

        int j;
    };
    int h = i;                                // 用 A::i 进行初始化
}
// 成员 f3 定义在 C1 和命名空间 A 的外部
int A::C1::f3() { return h; }                // 正确: 返回 A::h

```

除了类内部出现的成员函数定义之外（参见 7.4.1 节，第 254 页），总是向上查找作用域。名字必须先声明后使用，因此 f2 的 return 语句无法通过编译。该语句试图使用命名空间 A 的名字 h，但此时 h 尚未定义。如果 h 在 A 中定义的位置位于 C1 的定义之前，则上述语句将合法。类似的，因为 f3 的定义位于 A::h 之后，所以 f3 对于 h 的使用是合法的。



可以从函数的限定名推断出查找名字时检查作用域的次序，限定名以相反次序指出被查找的作用域。

限定符 A::C1::f3 指出了查找类作用域和命名空间作用域的相反次序。首先查找函数 f3 的作用域，然后查找外层类 C1 的作用域，最后检查命名空间 A 的作用域以及包含着 f3 定义的作用域。



实参相关的查找与类类型形参

考虑下面这个简单的程序：

```

std::string s;
std::cin >> s;

```

如我们所知，该调用等价于（参见 14.1 节，第 491 页）：

```

operator>>(std::cin, s);

```

798

operator>> 函数定义在标准库 string 中，string 又定义在命名空间 std 中。但是我们不用 std:: 限定符和 using 声明就可以调用 operator>>。

对于命名空间中名字的隐藏规则来说有一个重要的例外，它使得我们可以直接访问输出运算符。这个例外是，当我们给函数传递一个类类型的对象时，除了在常规的作用域查找外还会查找实参类所属的命名空间。这一例外对于传递类的引用或指针的调用同样有效。

在此例中，当编译器发现对 operator>> 的调用时，首先在当前作用域中寻找合适的函数，接着查找输出语句的外层作用域。随后，因为 >> 表达式的形参是类类型的，所以编译器还会查找 cin 和 s 的类所属的命名空间。也就是说，对于这个调用来说，编译器会查找定义了 istream 和 string 的命名空间 std。当在 std 中查找时，编译器找到了 string 的输出运算符函数。

查找规则的这个例外允许概念上作为类接口一部分的非成员函数无须单独的 using 声明就能被程序使用。假如该例外不存在，则我们将不得不为输出运算符专门提供一个 using 声明：

```

using std::operator>>;                      // 要想使用 cin >> s 就必须有该 using 声明

```

或者使用函数调用的形式以把命名空间的信息包含进来：

```

std::operator>>(std::cin, s);                // 正确: 显式地使用 std::>>

```

在没有使用运算符语法的情况下，上述两种声明都显得比较笨拙且无形中增加了使用 IO 标准库的难度。

查找与 `std::move` 和 `std::forward`

很多甚至是绝大多数 C++ 程序员从来都没有考虑过与实参相关的查找问题。通常情况下，如果在应用程序中定义了一个标准库中已有的名字，则将出现以下两种情况中的一种：要么根据一般的重载规则确定某次调用应该执行函数的哪个版本；要么应用程序根本就不会执行函数的标准库版本。

接下来考虑标准库 `move` 和 `forward` 函数。这两个都是模板函数，在标准库的定义中它们都接受一个右值引用的函数形参。如我们所知，在函数模板中，右值引用形参可以匹配任何类型（参见 16.2.6 节，第 611 页）。如果我们的应用程序也定义了一个接受单一形参的 `move` 函数，则不管该形参是什么类型，应用程序的 `move` 函数都将与标准库的版本冲突。`forward` 函数也是如此。

因此，`move`（以及 `forward`）的名字冲突要比其他标准库函数的冲突频繁得多。而且，因为 `move` 和 `forward` 执行的是非常特殊的类型操作，所以应用程序专门修改函数原有行为的概率非常小。

对于 `move` 和 `forward` 来说，冲突很多但是大多数是无意的，这一特点解释了为什么我们建议最好使用它们的带限定语的完整版本的原因（参见 12.1.5 节，第 417 页）。通过书写 `std::move` 而非 `move`，我们就能明确地知道想要使用的是函数的标准库版本。

799

友元声明与实参相关的查找



回顾我们曾经讨论过的，当类声明了一个友元时，该友元声明并没有使得友元本身可见（参见 7.2.1 节，第 242 页）。然而，一个另外的未声明的类或函数如果第一次出现在友元声明中，则我们认为它是最近的外层命名空间的成员。这条规则与实参相关的查找规则结合在一起将产生意想不到的效果：

```
namespace A {
    class C {
        // 两个友元，在友元声明之外没有其他的声明
        // 这些函数隐式地成为命名空间 A 的成员
        friend void f2();           // 除非另有声明，否则不会被找到
        friend void f(const C&);    // 根据实参相关的查找规则可以被找到
    };
}
```

此时，`f` 和 `f2` 都是命名空间 `A` 的成员。即使 `f` 不存在其他声明，我们也能通过实参相关的查找规则调用 `f`：

```
int main()
{
    A::C cobj;
    f(cobj);           // 正确：通过在 A::C 中的友元声明找到 A::f
    f2();              // 错误：A::f2 没有被声明
}
```

因为 `f` 接受一个类类型的实参，而且 `f` 在 `C` 所属的命名空间进行了隐式的声明，所以 `f` 能被找到。相反，因为 `f2` 没有形参，所以它无法被找到。

18.2.3 节练习

练习 18.18: 已知有下面的 `swap` 的典型定义 (参见 13.3 节, 第 457 页), 当 `mem1` 是一个 `string` 时程序使用 `swap` 的哪个版本? 如果 `mem1` 是 `int` 呢? 说明在这两种情况下名字查找的过程。

```
void swap(T v1, T v2)
{
    using std::swap;
    swap(v1.mem1, v2.mem1);
    // 交换类型 T 的其他成员
}
```

练习 18.19: 如果对 `swap` 的调用形如 `std::swap(v1.mem1, v2.mem1)` 将发生什么情况?

800

18.2.4 重载与命名空间

命名空间对函数的匹配过程有两方面的影响 (参见 6.4 节, 第 209 页)。其中一个影响非常明显: `using` 声明或 `using` 指示能将某些函数添加到候选函数集。另外一个影响则比较微妙。



与实参相关的查找与重载

在上一节中我们了解到, 对于接受类类型实参的函数来说, 其名字查找将在实参类所属的命名空间中进行。这条规则对于我们如何确定候选函数集同样也有影响。我们将在每个实参类 (以及实参类的基类) 所属的命名空间中搜寻候选函数。在这些命名空间中所有与被调用函数同名的函数都将被添加到候选集当中, 即使其中某些函数在调用语句处不可见也是如此:

```
namespace NS {
    class Quote { /* ... */ };
    void display(const Quote&) { /* ... */ }
}
// Bulk_item 的基类声明在命名空间 NS 中
class Bulk_item : public NS::Quote { /* ... */ };
int main() {
    Bulk_item book1;
    display(book1);
    return 0;
}
```

我们传递给 `display` 的实参属于类类型 `Bulk_item`, 因此该调用语句的候选函数不仅应该在调用语句所在的作用域中查找, 而且也应该在 `Bulk_item` 及其基类 `Quote` 所属的命名空间中查找。命名空间 `NS` 中声明的函数 `display(const Quote&)` 也将被添加到候选函数集当中。

重载与 `using` 声明

要想理解 `using` 声明与重载之间的交互关系, 必须首先明确一条: `using` 声明语句声明的是一个名字, 而非一个特定的函数 (参见 15.6 节, 第 551 页):

```
using NS::print(int);    // 错误: 不能指定形参列表
using NS::print;         // 正确: using 声明只声明一个名字
```

当我们为函数书写 using 声明时，该函数的所有版本都被引入到当前作用域中。

一个 using 声明囊括了重载函数的所有版本以确保不违反命名空间的接口。库的作者为某项任务提供了好几个不同的函数，允许用户选择性地忽略重载函数中的一部分但不是全部有可能导致意想不到的程序行为。

一个 using 声明引入的函数将重载该声明语句所属作用域中已有的其他同名函数。如果 using 声明出现在局部作用域中，则引入的名字将隐藏外层作用域的相关声明。如果 using 声明所在的作用域中已经有一个函数与新引入的函数同名且形参列表相同，则该 using 声明将引发错误。除此之外，using 声明将为引入的名字添加额外的重载实例，并最终扩充候选函数集的规模。

◀ 801

重载与 using 指示

using 指示将命名空间的成员提升到外层作用域中，如果命名空间的某个函数与该命名空间所属作用域的函数同名，则命名空间的函数将被添加到重载集合中：

```
namespace libs_R_us {
    extern void print(int);
    extern void print(double);
}
// 普通的声明
void print(const std::string &);
// 这个 using 指示把名字添加到 print 调用的候选函数集
using namespace libs_R_us;
// print 调用此时的候选函数包括：
// libs_R_us 的 print(int)
// libs_R_us 的 print(double)
// 显式声明的 print(const std::string &)
void fooBar(int ival)
{
    print("Value: ");           // 调用全局函数 print(const string &)
    print(ival);                // 调用 libs_R_us::print(int)
}
```

与 using 声明不同的是，对于 using 指示来说，引入一个与已有函数形参列表完全相同的函数并不会产生错误。此时，只要我们指明调用的是命名空间中的函数版本还是当前作用域的版本即可。

跨越多个 using 指示的重载

如果存在多个 using 指示，则来自每个命名空间的名字都会成为候选函数集的一部分：

```
namespace AW {
    int print(int);
}
namespace Primer {
    double print(double);
}
// using 指示从不同的命名空间中创建了一个重载函数集合
using namespace AW;
using namespace Primer;
long double print(long double);
int main() {
```

◀ 802

```

    print(1);                // 调用 AW::print(int)
    print(3.1);              // 调用 Primer::print(double)
    return 0;
}

```

在全局作用域中，函数 `print` 的重载集合包括 `print(int)`、`print(double)` 和 `print(long double)`，尽管它们的声明位于不同作用域中，但它们都属于 `main` 函数中 `print` 调用的候选函数集。

18.2.4 节练习

练习 18.20: 在下面的代码中，确定哪个函数与 `compute` 调用匹配。列出所有候选函数和可行函数，对于每个可行函数的实参与形参的匹配过程来说，发生了哪种类型转换？

```

namespace primerLib {
    void compute();
    void compute(const void *);
}
using primerLib::compute;
void compute(int);
void compute(double, double = 3.4);
void compute(char*, char* = 0);
void f()
{
    . compute(0);
}

```

如果将 `using` 声明置于 `main` 函数中 `compute` 的调用点之前将发生什么情况？重新回答之前的那些问题。

18.3 多重继承与虚继承

多重继承 (multiple inheritance) 是指从多个直接基类 (参见 15.2.2 节, 第 533 页) 中产生派生类的能力。多重继承的派生类继承了所有父类的属性。尽管概念上非常简单, 但是多个基类相互交织产生的细节可能会带来错综复杂的设计问题与实现问题。

为了探讨有关多重继承的问题, 我们将以动物园中动物的层次关系作为教学实例。动物园中的动物存在于不同的抽象级别上。有个体的动物, 如 `Ling-Ling`、`Mowgli` 和 `Balou` 等, 它们以名字进行区分; 每个动物属于一个物种, 例如 `Ling-Ling` 是一只大熊猫; 物种又是科的成员, 大熊猫是熊科的成员; 每个科是动物界的成员, 在这个例子中动物界是指一个动物园中所有动物的总和。

我们将定义一个抽象类 `ZooAnimal`, 用它来保存动物园中动物共有的信息并提供公共接口。类 `Bear` 将存放 `Bear` 科特有的信息, 以此类推。

除了类 `ZooAnimal` 之外, 我们的应用程序还包含其他一些辅助类, 这些类负责封装不同的抽象, 如濒临灭绝的动物。以类 `Panda` 的实现为例, `Panda` 是由 `Bear` 和 `Endangered` 共同派生而来的。